SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	HARI VISHNU N	**Reg. No.:**	192524319
Date:	24-10-2026	**Duration:**	60 minutes

Code of Conduct

I **HARI VISHNU N / 192524319** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: 

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an individual software project problem statement. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your assigned problem statement, perform an Architecture Design Analysis and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

QUESTION:

AI-Powered Requirements Conflict and Ambiguity Detection System

Assigned Problem Statement (No. 11)

Industry Context: Large software projects receive requirements from users, customers, business teams, developers, security teams, and regulators. Requirements may be incomplete, ambiguous, duplicated, or contradictory. These problems can remain hidden until design or implementation, resulting in costly rework.

Problem Statement: Design and develop an AI-Powered Requirements Conflict and Ambiguity Detection System that analyses software requirements and identifies inconsistencies, ambiguity, duplication, missing dependencies, and conflicting stakeholder expectations. The system should support collaborative documentation, requirements prioritization, traceability, and human validation. It should integrate with project-management tools and maintain version history while ensuring that AI recommendations remain reviewable and explainable.

Objectives: The system should:

- Analyse requirements for ambiguity and contradictions
- Detect duplicate and conflicting requirements
- Identify missing dependencies and incomplete information
- Support MoSCoW or similar prioritization techniques
- Maintain traceability between requirements and development tasks
- Provide collaborative review and version control
- Support explainable and human-validated AI recommendations

Expected Outcomes: The proposed system will reduce requirements-related rework, improve clarity and consistency of specifications, support stakeholder collaboration, improve requirements traceability, reduce project risks caused by conflicting requirements, and strengthen requirements engineering practices.

1 Analyze System Requirements

> **1. Analyze System Requirements**
>
> **Study the given problem statement and identify the system objectives. Analyze the functional and non-functional requirements that influence the software architecture. Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.**

1.1 System Objectives

The problem statement describes a system that analyses requirements and exposes defects before they reach design or implementation. The objectives can be grouped into four areas:

- **Detection:** find ambiguous, contradictory, duplicated, and incomplete requirements, along with missing dependencies.
- **Prioritisation:** support MoSCoW and similar techniques so stakeholders can agree on must-have, should-have, could-have, and won't-have items.
- **Traceability:** link each requirement to development tasks, tests, and design elements, and keep those links current.
- **Governance:** provide collaborative review, version history, and human validation, so that AI output is reviewed and explained rather than accepted automatically.

Reading the statement closely, several points are not written as features but still shape the design: the AI must be explainable, humans keep final authority over every change, the system must integrate with existing project-management tools rather than replace them, and history must be preserved so that every decision can be audited.

1.2 Stakeholders

Stakeholder	Main Interest	Key Concern
Business analysts / requirements engineers	Clear and consistent specifications	Review effort and false-positive warnings
Customers and end users	Requirements reflect their real needs	Their input being lost or overridden

Stakeholder	Main Interest	Key Concern
Developers and testers	Stable, unambiguous requirements	Rework caused by conflicts found late
Security team and regulators	Security and compliance requirements are present and consistent	Missing or conflicting security and compliance rules
Project managers	Predictable scope and schedule	Visibility of risk and priority disagreements
System administrators	Secure, maintainable operation	Integration effort and protection of data

1.3 Functional Requirements

ID	Functional Requirement	MoSCoW
FR1	Capture and import requirements manually, from CSV/Excel, ReqIF, and project-management tools	Must
FR2	Detect ambiguity: vague terms (e.g., 'fast', 'user-friendly'), passive voice, missing actor, unmeasurable quantities	Must
FR3	Detect duplicate and near-duplicate requirements using semantic similarity	Must
FR4	Detect conflicting requirements: direct contradictions and conflicts over shared constraints or resources	Must
FR5	Identify missing dependencies and incomplete information (no acceptance criteria, unlinked NFRs, undefined actors)	Must
FR6	Support MoSCoW prioritisation with stakeholder voting and weighting	Should
FR7	Maintain traceability between requirements, tasks, tests, and design items, with impact analysis	Must

ID	Functional Requirement	MoSCoW
FR8	Collaborative review: comments, @mentions, reviewer assignment, accept / reject / edit of AI findings	Must
FR9	Version control: history, differences between versions, baselines, and rollback	Must
FR10	Two-way synchronisation with Jira, Azure DevOps, and GitHub Issues	Should
FR11	Explain every AI finding: triggering text, rule or model rationale, and confidence score	Must
FR12	Dashboards showing ambiguity density, open conflicts, and rework indicators	Could

1.4 Non-Functional Requirements and Their Architectural Impact

Quality Attribute	Requirement / Target	Why It Matters for Architecture
Scalability	500 projects, 50,000 requirements per project, 200 concurrent users; analysis workers scale out	Pairwise comparison grows quickly with requirement count, so analysis compute must scale on its own
Performance	Feedback on an edited requirement within 5 s (95th percentile); full re-analysis of 10,000 requirements within 15 minutes	Editing must feel interactive; heavy analysis must run in the background
Reliability	No saved requirement version is ever lost; failed analysis jobs are retried, not dropped	Requirements are long-lived project records and may be contractual
Availability	99.5% monthly uptime for core editing; AI services may degrade without blocking editing	Teams must keep working even when AI components are unavailable

Quality Attribute	Requirement / Target	Why It Matters for Architecture
Security	SSO login, role-based access, TLS 1.2+ in transit, AES-256 at rest, audit logs; no requirement text sent to external AI services without approval	Requirements contain confidential and regulated information
Maintainability	Modular services with stable interfaces; at least 80% unit-test coverage; new detector or connector added without changing core code	Detection rules and models change frequently
Explainability and auditability	Every AI finding includes evidence and rationale and is logged with the reviewer's decision	Needed for human validation and for trust in the tool
Usability	Common review actions completed in three clicks or fewer; keyboard-accessible	Adoption depends on low effort per review

1.5 Key Quality Attributes and Their Priority

The six attributes named in the question do not carry equal weight for this system. Ranked by their influence on the architecture, the order is:

- **Security and confidentiality:** requirements often describe unreleased products, security controls, and regulated data, so exposure would be the most serious failure.
- **Reliability and maintainability:** requirement records must survive for the full project lifecycle, and the detection logic will change as the team learns which findings matter.
- **Scalability and performance:** conflict checking compares candidate pairs, so the cost rises much faster than the number of requirements.
- **Availability:** important, but the core requirement-editing function can be kept running even when the AI services are degraded.

An analysis insight also applies. The expected outcome of reduced rework depends on **precision**. A tool that flags too many non-issues will be ignored by reviewers, so explanation, confidence scores, and reviewer feedback are design concerns as well as user-experience concerns.

2 Select a Suitable Software Architecture

> **2. Select a Suitable Software Architecture**
>
> **Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture). Justify why the selected architecture is the most suitable for the given problem.**

2.1 Selected Style: Hybrid (Layered Core with an Event-Driven AI Pipeline)

The system has two workloads that behave very differently. The first is interactive, transactional collaboration, which needs immediate and consistent responses when a requirement is edited, approved, or linked. The second is analysis: embedding generation, similarity search, and model inference take seconds to minutes per batch and should run without blocking the user. I therefore selected a **hybrid architecture** that uses a **layered architecture** for the core product (presentation, API, application services, and data) and an **event-driven pipeline** for the AI analysis.

2.2 Comparison of Candidate Architectural Styles

Style	Strengths for This Problem	Weaknesses for This Problem	Fit
Layered	Simple, clear separation of concerns, straightforward transactions	Analysis jobs share scaling with the whole application	Partial
Client-Server	Simple interaction model	No separation for heavy analysis; weak modularity	Low
MVC	Good structure for the user interface	Says little about back-end scaling or AI processing	Low
Microservices	Independent scaling and deployment of each capability	High operational overhead for a first version; cross-service traceability and version queries are complex	Medium

Style	Strengths for This Problem	Weaknesses for This Problem	Fit
Event-Driven	Natural fit for asynchronous, decoupled detectors	Harder to keep consistent; users need synchronous responses for edits and approvals	Partial alone
Service-Oriented (SOA)	Good for enterprise integration	Heavy governance; slows experimentation with AI components	Medium
Serverless	Cost-efficient for bursty workloads	Long-running NLP jobs, cold starts, vendor lock-in, and difficulty supporting on-premise deployment	Low
Hybrid (Layered + Event-Driven)	Consistent core data; asynchronous and independently scalable AI workers; can start as a modular deployment and split later	Needs strict module boundaries and two communication styles	**High**

2.3 Justification of the Selection

- **Consistency of requirements data:** versions, baselines, approvals, and traceability links must be written atomically. A layered core with one relational database makes this simple and reliable.
- **Asynchronous and scalable analysis:** a change event triggers detection, which can run in the background. When many requirements change at once, only the analysis worker pool grows.
- **Pluggable detectors:** each detector (ambiguity, duplicate, conflict, completeness) consumes the same events and returns findings in the same format. A new detector is added without changing the core.

- **Integration isolation:** project-management tools differ in their APIs and change them over time. Connectors sit behind an integration layer, so the core domain model is not affected.
- **Pragmatic evolution:** the core can be deployed as a modular unit first, while the analysis workers are deployed separately from the start. This avoids the operational cost of full microservices.

Why not pure microservices? A team building the first version would carry the cost of many deployable services, and queries such as "show every task linked to requirements that changed in version 4" would cross several service boundaries. **Why not pure event-driven?** Reviewers need immediate feedback on their own actions, and approvals must be visible to everyone straight away. The hybrid keeps those interactions synchronous and uses events only where delay is acceptable.

3 Draw the Software Architecture Diagram

> **3. Draw the Software Architecture Diagram**
>
> **Design a clear architecture diagram illustrating the overall structure of the proposed system. Include all major layers/components, databases, external systems, APIs, and communication mechanisms. Use appropriate software architecture notations and labels.**

3.1 Architecture Diagram

Figure 1 shows the system as six horizontal zones: external systems, the presentation layer, the API gateway, the application services, the AI analysis pipeline, and the data layer. The pipeline is shown as a separate zone because it is event-driven, while the rest of the system is layered.

3.2 Notation Used

Symbol	Meaning
Rounded box with navy border	Component or deployable unit
Dashed-border zone	Architectural layer or subsystem boundary
Grey box	External system or dependency outside the product
Double-headed solid arrow	Synchronous request and response (REST or WebSocket)
Dashed arrow	External or optional connection (for example, identity provider login)
Single-headed arrow inside the AI pipeline	Asynchronous event flow through the event broker

To read the diagram, start at the top. Stakeholders reach the web client, which calls the API gateway. The gateway routes each request to the correct application service. Services that change requirements publish events to the broker, the orchestrator runs the detectors, and the explanation engine stores the findings. Every component that persists data uses the data layer.

4 Explain Components and Their Interactions

> **4. Explain Components and Their Interactions**
>
> **Describe the purpose and responsibilities of each architectural component. Explain how the components communicate and exchange data. Illustrate the overall workflow of the system.**

4.1 Purpose and Responsibilities of Each Component

Component	Layer	Responsibilities	Main Interface
Web Client (SPA)	Presentation	Requirement editor with inline highlights, review queue, MoSCoW board, dashboards	REST, WebSocket
PM Tool Connectors	Presentation / Integration	Plug-ins that display and synchronise tasks with Jira, Azure DevOps, and GitHub	Vendor APIs, webhooks
API Gateway	Gateway	Single entry point; validates tokens, enforces RBAC, applies rate limits, routes requests, receives webhooks	HTTPS REST, WebSocket
Requirements Management Service	Application	Create, read, update, and import requirements; create immutable versions; manage baselines	REST; publishes RequirementChanged
Collaboration & Review Service	Application	Comments, mentions, reviewer assignment, approval workflow, recording of decisions on findings	REST, WebSocket
Prioritisation Service	Application	MoSCoW classification, stakeholder voting, weighting, and highlighting of disagreement	REST
Traceability Service	Application	Maintains links between requirements, tasks, tests, and design items; produces impact analysis	REST, graph queries
Integration Sync Service	Application	Schedules two-way synchronisation, maps fields, resolves sync conflicts	Webhooks, connector queue

Component	Layer	Responsibilities	Main Interface
Notification & Access Control	Application	Sends review requests and alerts; manages users, roles, and project permissions	Email, chat webhooks, SSO
Event Broker	AI pipeline	Durable topics for change and analysis events, with retries and dead-letter queues	Kafka protocol
Analysis Orchestrator	AI pipeline	Consumes changes, batches related requirements, runs detectors in parallel, merges and de-duplicates results	Internal events
Ambiguity Detector	AI pipeline	Rule checks for vague terms, passive voice, and missing measurable criteria; NLP model for unclear references	Internal events
Duplicate Detector	AI pipeline	Sentence embeddings and cosine similarity to find near-duplicate requirements	Vector store
Conflict Detector	AI pipeline	Checks candidate pairs for contradiction using an NLI model, plus rule checks on numeric limits	NLP / LLM service
Completeness & Dependency Checker	AI pipeline	Checks for actors, acceptance criteria, missing dependencies, and orphan NFRs	Requirements data, dependency graph
Explanation & Confidence Engine	AI pipeline	Attaches evidence, rationale, severity, and confidence to each finding; sets status to 'Pending human review'	Findings store
PostgreSQL	Data	Authoritative store for requirements, versions, workflow state, users, and decisions	SQL

Component	Layer	Responsibilities	Main Interface
Vector Store (pgvector)	Data	Stores embeddings and supports similarity search	SQL with vector index
Object Store	Data	Attachments and generated exports	S3-compatible API
Audit & Analytics Store	Data	Append-only record of changes and decisions; source for dashboards	Append-only tables, analytics export

4.2 Communication Mechanisms and Data Exchanged

From	To	Mechanism	Data Exchanged
Web Client	API Gateway	HTTPS / JSON; WebSocket for live editing	Requirements, comments, review actions
API Gateway	Application Services	Internal HTTP / JSON with validated bearer token	Requests with verified user context
Requirements Service	Event Broker	Asynchronous publish	RequirementChanged: id, version, text, project
Event Broker	Orchestrator	Consumer-group subscription; at-least-once delivery	Change events; redelivered on failure
Orchestrator	Detectors	Internal events	AnalysisRequested: batch and detector type
Detectors	Vector Store / NLP service	Internal calls	Embeddings, candidate pairs, text for entailment checks
Detectors	Explanation Engine	Internal events	Raw findings with supporting evidence

From	To	Mechanism	Data Exchanged
Explanation Engine	PostgreSQL	Transactional write	Findings with status 'Pending human review'
Collaboration Service	PostgreSQL	Transaction	Accept / reject decision, reviewer, timestamp, reason
Integration Sync	PM Tools	REST and webhooks	Task creation, status, field mapping
All services	Audit Store	Event	Immutable audit record
Identity Provider	API Gateway	OIDC redirect and token validation	Identity and group claims

4.3 Overall System Workflow

The following workflow follows one requirement from creation to traceable, reviewed status:

Step 1 – Capture: a business analyst creates or imports a requirement. The Requirements Service stores a new immutable version and publishes a RequirementChanged event.

Step 2 – Analyse: the orchestrator collects the changed requirement together with its nearest neighbours (identified through embeddings) and triggers the four detectors in parallel.

Step 3 – Explain: the Explanation Engine attaches evidence, rationale, severity, and confidence to each result. Findings are saved as 'Pending human review' and the author is notified.

Step 4 – Review: a reviewer opens the review queue and accepts, rejects, or edits each finding. The decision and its reason are recorded, and rejected findings are used to tune detection thresholds.

Step 5 – Prioritise: stakeholders vote on MoSCoW categories. The Prioritisation Service records the votes and highlights disagreements for discussion.

Step 6 – Trace and synchronise: accepted requirements are linked to tasks, and the Integration Sync Service creates or updates the corresponding Jira, Azure DevOps, or GitHub items. Status changes return through webhooks.

Step 7 – Report: dashboards read quality metrics from the audit and analytics store.

A key design rule runs through this workflow: **the AI never changes requirement text by itself.** Every modification is a human decision, which keeps the requirements record accountable.

5 Discuss Quality Attributes

> **5. Discuss Quality Attributes**
>
> **Evaluate how the proposed architecture addresses the following aspects: Scalability, Security, Performance, Reliability, Maintainability, Availability.**

5.1 Scalability

The architecture scales in three ways. Application services are stateless behind the gateway, so more instances can be added behind a load balancer. Analysis workers scale according to the length of their queue, and broker topics are partitioned by project so that work for different projects runs in parallel.

The most important scalability measure is **candidate-pair pruning**. Checking every requirement against every other one requires $n(n-1)/2$ comparisons. For 10,000 requirements that is about 50 million pairs. Duplicate and conflict detection only compares pairs whose embedding similarity exceeds a threshold, so the set of expensive model calls becomes a small fraction of that total. Analysis is also incremental: when a requirement changes, only that requirement and its neighbours are re-analysed.

Residual risk: the vector index needs memory in proportion to the number of embeddings. This is controlled by partitioning per project and, at very large scale, by moving the index to a dedicated vector store.

5.2 Security

- **Authentication:** single sign-on through OpenID Connect with multi-factor authentication enforced by the identity provider.
- **Authorisation:** role-based access (Author, Reviewer, Stakeholder, Administrator, Auditor), applied per project.
- **Encryption:** TLS 1.2 or higher in transit and AES-256 at rest. Secrets such as connector tokens are kept in a managed vault, never in source code.
- **Data protection:** the NLP and LLM services can be deployed inside the organisation's network, so requirement text does not leave it. Logs store identifiers rather than full requirement text.
- **Input safety:** imported requirement text is treated as data, never as instructions to the model, which protects against crafted content inside an imported file.

- **Integrity and accountability:** the audit store is append-only, and connector tokens have the minimum permissions needed.

5.3 Performance

Performance rests on keeping user-facing operations separate from heavy analysis. Saving a requirement returns in under one second because it only writes to the database and publishes an event. Findings then appear as the pipeline completes them, typically within seconds for a single change.

- **Cheap checks first:** rule-based ambiguity checks run before any model call, and model calls are made only for uncertain cases.
- **Caching:** embeddings are cached and recomputed only when a requirement's text changes.
- **Parallel detection:** the four detectors run concurrently, so total latency is set by the slowest detector rather than their sum.
- **Targets:** feedback on a single edited requirement within 5 seconds at the 95th percentile; full analysis of 10,000 requirements within 15 minutes.
- **Paginated review queue:** reviewers see findings in pages, so large projects do not overload the browser.

5.4 Reliability

- **Durable messaging:** the event broker persists events. Consumers commit their offsets only after processing, so no change is lost if a worker fails.
- **Retries and dead-letter queues:** failed jobs are retried with exponential backoff. Messages that keep failing move to a dead-letter queue for inspection.
- **Idempotent processing:** findings are keyed by requirement version, detector, and input hash, so a redelivered event produces the same result instead of duplicates.
- **Immutable versions:** every save creates a new version and no record is overwritten, so history is never lost.
- **Fault isolation:** if one detector fails, the others continue and the result is marked as partial.

5.5 Maintainability

- **Clear boundaries:** each service owns its own data and exposes a versioned API, so internal changes do not break other services.
- **Common detector contract:** every detector accepts a set of requirements and returns findings in one schema. Adding a detector means adding a consumer, not changing the orchestrator.
- **Connector pattern:** each project-management tool sits behind an adapter that implements a single interface.
- **Externalised configuration:** thresholds and rule sets are stored as configuration and tuned from reviewer feedback, without redeploying code.
- **Testing and delivery:** unit tests, contract tests for event schemas, and end-to-end tests run in a continuous-integration pipeline.
- **Observability:** structured logs, metrics, and distributed tracing show where time is spent and where failures occur.

5.6 Availability

The target is **99.5% availability** for core editing, which allows about 3.6 hours of downtime in a 730-hour month. The design achieves this through redundancy and graceful degradation:

- Each service runs as at least two instances across two availability zones, behind a load balancer with health checks.
- The database uses synchronous replication with automatic failover.
- If the AI pipeline is unavailable, requirement editing and review continue normally. Change events remain in the broker, and the analysis resumes when the pipeline recovers. The interface shows a 'degraded analysis' notice.
- Rolling deployments allow updates without downtime.

5.7 Summary of Quality Attribute Mechanisms

Quality Attribute	Main Architectural Mechanism	Residual Risk
Scalability	Stateless services, queue-based workers, candidate-pair pruning	Vector index memory at very large scale

Quality Attribute	Main Architectural Mechanism	Residual Risk
Security	SSO, RBAC, encryption, on-premise model option, append-only audit	Misconfigured connector permissions
Performance	Asynchronous analysis, cheap checks first, caching, parallel detectors	Model latency on slow hardware
Reliability	Durable broker, retries, dead-letter queues, immutable versions	Dependence on broker configuration
Maintainability	Module boundaries, detector contract, connector adapters, CI pipeline	Growing number of rules to maintain
Availability	Redundancy, failover, graceful degradation	Dependence on external PM tool availability

6 Justify Architectural Decisions

> **6. Justify Architectural Decisions**
>
> **Explain the rationale behind your architectural choices. Discuss the trade-offs considered while selecting the architecture. Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.**

6.1 Rationale for Key Decisions

Decision	Rationale	Trade-off Accepted
D1: Hybrid architecture instead of full microservices	Small initial team; versioning and approvals need consistent transactions	Core services cannot be deployed independently; module boundaries must be enforced by discipline
D2: Event-driven AI pipeline	Analysis is long-running and bursty; detectors must be decoupled	Eventual consistency: findings appear seconds after a save
D3: Hybrid detection (rules plus NLP / LLM models)	Rules are cheap, deterministic, and easy to explain; models catch subtle meaning	Two methods to build and maintain
D4: Human-in-the-loop by design	Requirements are contractual; wrong automatic changes are costly	Slower acceptance; the review backlog needs management
D5: PostgreSQL with pgvector	One database for transactional data and similarity search reduces operational components	Very large embedding sets may later need a dedicated vector store
D6: Explainable findings with evidence	Required for trust and audit	Extra storage and less freedom in model choice
D7: Connectors behind an integration layer	Project-management tools differ and change their APIs	Ongoing maintenance for each tool

6.2 Trade-offs Considered

Trade-off	Option A	Option B	Decision
Consistency vs. responsiveness	Strong consistency everywhere	Eventual consistency for analysis results	Strong consistency for requirement data; eventual for findings, with a visible 'analysis in progress' state
Accuracy vs. explainability	Large black-box model	Transparent rules only	Hybrid: rules give direct explanations; models give evidence sentences and confidence
Cloud vs. on-premise	Cloud hosting: cheaper and simpler	On-premise: stronger confidentiality	Containerised services and configurable model endpoints, so both are possible
Recall vs. precision	Flag everything	Flag only high-confidence issues	Configurable thresholds; reviewer 'false positive' feedback adjusts them over time
Analysis cost vs. coverage	Compare all pairs	Compare only likely pairs	Embedding-based pruning, caching, and incremental analysis

The most significant trade-off is **accuracy versus explainability**. A large language model can detect subtle contradictions that rules miss, but a reviewer who sees only a verdict cannot judge it. The design keeps both: deterministic rules provide direct explanations, and model-based findings always include the sentences that triggered them and a confidence value. Reviewers can therefore see why a finding exists and decide whether to accept it.

6.3 Support for Future Enhancements, Integration, and Maintainability

- **New detectors:** a regulatory-compliance checker, for example against a privacy standard, can be added as another consumer of the same events without touching the core.
- **New integrations:** further tools such as Confluence or Polarion can be added as connectors that implement the existing adapter interface.
- **Multi-language requirements:** language-specific NLP models can be configured per project behind the same detector contract.
- **Model upgrades:** models sit behind an endpoint abstraction. A better model can replace an older one, and a fixed evaluation set measures whether precision has improved before the change is released.
- **Scale-out:** a service can be moved into its own deployment when load requires it. Its boundary and data ownership are already defined, so the change is incremental.
- **Long-term records:** versioned APIs and event schemas, retained reviewer feedback for retraining, and an immutable audit history all support long-running projects and external audits.

In conclusion, the hybrid architecture gives the system a consistent and secure core for requirements data, while keeping the computationally heavy and rapidly evolving AI analysis independent. This matches the problem's main demands: trustworthy requirement records, explainable AI output, human validation, and integration with existing project-management tools.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25